

Hush-Hub: A Web Application for Discovering Quiet Workspaces in Manhattan

Jianqiao Xu

University College Dublin
Dublin, Ireland
jianqiao.xu@ucdconnect.ie

Jason Lee

University College Dublin
Dublin, Ireland
jason.lee@ucdconnect.ie

Yangxiaoshi Gao

University College Dublin
Dublin, Ireland
yangxiaoshi.gao@ucdconnect.ie

Jialin Wang

University College Dublin
Dublin, Ireland
jialin.wang1@ucdconnect.ie

Ziling Huang

University College Dublin
Dublin, Ireland
ziling.huang@ucdconnect.ie

Zachary Gillespie

University College Dublin
Dublin, Ireland
zachary.gillespie@ucdconnect.ie

Abstract—Finding a quiet place to work or study in a dense urban environment such as Manhattan, New York City, is a persistent and underserved challenge. Existing venue-discovery platforms rank spaces primarily by popularity or cuisine, providing little actionable information about noise, crowding, or temporal busyness patterns. We present *Hush-Hub*, a full-stack web application that aggregates and analyses multiple independent environmental signals to help users identify suitable quiet workspaces across 326 curated venues. The system integrates four complementary data dimensions: indoor quietness derived from editorial reviews, ambient street noise estimated from road classifications and NYC 311 noise complaint density, transient disruption from active construction permits and permitted loud events, and area foot-traffic busyness modelled from MTA subway ridership using distance-weighted Gaussian decay. A reproducible seven-stage pipeline turns editorial articles into structured, evidence-backed venue records, and a random-forest ridership predictor reduces mean absolute error by 19% relative to a strong temporal baseline. The signals are served through a FastAPI REST API and consumed by a React map-centric web frontend and a Flutter mobile companion, following the broader data-driven use of open urban data demonstrated in [1]. A conversational AI assistant enables natural-language venue discovery, and the interface supports English, Chinese and Spanish.

I. INTRODUCTION

The rise of remote and hybrid work has intensified demand for accessible, quiet spaces outside the home. Coffee shops, libraries, and co-working venues have absorbed much of this demand, serving as “third places” that bridge domestic and professional environments [2]. Yet these spaces are not always suitable for focused work: environmental noise, crowding, and transient disruptions such as construction or nearby events can severely degrade productivity and wellbeing [3], [4].

Mainstream venue-discovery platforms such as Google Maps, Yelp, and Foursquare rank and filter spaces by popularity, user rating, or category. While these signals are useful for restaurant selection, they are poor proxies for quietness or suitability for focused work. Star ratings aggregate enjoy-

ment across all use cases; a lively neighbourhood bar scores well for social visits but poorly for deep work. Location-based recommendation systems have explored social influence, geospatial patterns, and contextual features, but they rarely treat acoustic comfort or temporal busyness as first-class search dimensions [5], [6].

This paper addresses the following problem: *How can we identify and recommend quiet urban workspaces by fusing heterogeneous data sources—editorial reviews, civic noise complaints, transit ridership, and event permits—into interpretable, time-varying quietness indicators?* To answer this, we present *Hush-Hub*, a full-stack web application targeting Manhattan, New York City—a borough of exceptional venue density and noise complexity. The system curates 326 candidate venues (cafes, libraries, restaurants, bars, and public study areas) and enriches each with four orthogonal signals: (i) indoor quietness inferred from editorial crowdsourcing, (ii) street-level noise from road classifications and reported 311 complaints, (iii) transient disruptions from construction and loud-event permits, and (iv) hourly area busyness derived from MTA subway ridership. These signals are deliberately kept separate and expressed in absolute, interpretable units rather than collapsed into a single opaque score.

II. LITERATURE REVIEW

A. Urban Noise, Quiet Areas, and Health

Environmental noise is now recognised as one of the most pervasive environmental stressors in urban areas. The World Health Organization’s 2018 guidelines link long-term noise exposure to sleep disturbance, cardiovascular risk, and reduced cognitive performance in children and adults [3]. Murphy and King provide a systematic treatment of noise mapping, public-health impacts, and planning policy, showing that noise assessment must move beyond simple decibel contours toward spatially explicit, context-sensitive models [4]. Empirical studies of quiet-area identification have combined field measure-

ments, soundscape recordings, and strategic noise mapping to delineate urban quiet areas [7], [8]. These works establish that quietness is both an acoustic property and a perceived quality, and that effective quiet-area mapping requires multiple data sources rather than single-indicator proxies.

B. Location-Based Recommendation and Venue Discovery

Location-based social networks have motivated extensive research on venue recommendation. Bao et al. survey collaborative-filtering, content-based, and hybrid methods that exploit user check-ins, spatial proximity, and social ties [5]. Ye et al. demonstrate how social influence can be incorporated into generative models for point-of-interest recommendation [6]. However, these systems typically optimise for popularity, novelty, or social relevance, not for the task-specific suitability of a venue for focused work. They also lack the temporal busyness and environmental-noise signals that determine whether a space is currently quiet.

C. Existing Quiet-Workspace Applications

Several products attempt to help users find quiet study or work spaces. *NOOQ Spot* scores locations by quietness and lists amenities such as Wi-Fi and power outlets, but its data depend heavily on user-generated content and it does not provide a visual map for discovery or comparison. *Study Spot* targets students with a simple venue directory, but it offers limited information per location and is not designed for office workers or tourists. Hush City crowdsources quiet-area reports through citizen science, producing valuable soundscape data, yet coverage is uneven and it does not predict future busyness or personalise recommendations [9]. By contrast, Google Maps and Foursquare provide broad coverage and rich metadata, but their ranking signals reflect general popularity rather than suitability for focused work.

D. Gap Analysis

The literature and existing products reveal three gaps that Hush-Hub addresses. First, acoustic comfort and temporal busyness are rarely treated as primary recommendation features. Second, existing quiet-space apps rely on sparse crowdsourcing or narrow user segments rather than on systematically fused civic and editorial data. Third, no widely available tool gives users interpretable, multi-source indicators that explain why a venue is quiet or busy at a particular time. Hush-Hub fills these gaps by combining editorial reviews, NYC 311 noise complaints, MTA ridership, and event permits into a unified, time-aware recommendation interface for quiet workspaces.

III. METHODOLOGY

A. User Research and Personas

Requirements were gathered through semi-structured interviews with representative users from three target groups in Manhattan: university students, remote workers, and tourists. The interviews focused on how each group selects a workspace outside the home, what environmental factors they consider, and what tools they currently use. From these findings, we

developed three personas and mapped their goals to concrete user stories with acceptance criteria.

1) Personas:

- **University Student:** Needs free, quiet, low-occupancy study spots near campus, with the ability to compare options and save favourites.
- **Remote Office Worker:** Needs weather-aware workspaces with reliable Wi-Fi, transit access, and route guidance.
- **Tourist:** Needs visual map guidance, photos, and intuitive navigation in an unfamiliar city.

2) *User Stories and Acceptance Criteria:* Table I summarises the user stories and acceptance criteria derived from the interviews. The criteria directly shaped the product features described in Section III-B.

TABLE I
USER STORIES AND ACCEPTANCE CRITERIA

Persona	User Story	Acceptance Criteria
Student	As a student, I want to find quiet, low-occupancy study places near campus, compare candidates, and save regular spots so I can work efficiently.	Show Quiet Score, occupancy, and distance; filter by venue type; sort by score/distance; current and predicted Quiet Scores; bookmark and view favourites; tags and ratings; comparison; smart recommendations based on time and weather.
Office Worker	As a remote worker, I want weather-aware workspace recommendations with reliable Wi-Fi and transit info so I can work productively outside home.	Weather panel with smart tips; venue photos; Google Maps route link; detail panel shows Wi-Fi, hours, tags, reviews, and ratings.
Tourist	As a tourist unfamiliar with Manhattan, I want to find quiet places near my hotel via an intuitive map and visual cues, with friendly guidance when filters are too strict.	Color-coded markers by venue type; photo carousels in cards and detail view; animated empty state with tips; recently viewed list.

B. Product Design

1) *Design Overview:* Hush-Hub is a single-page, map-centric web application for discovering quiet study and work spaces in Manhattan. The interface follows a three-column layout: a left sidebar for search and discovery, a central interactive map for spatial exploration, and a right panel for venue details and chat. The design emphasises focus, clarity, and speed; translucent glassmorphism panels and subtle motion reduce cognitive load while keeping key information accessible.

2) *Information Architecture:* Core entities include *Venue* (id, name, address, type, coordinates, Quiet Score, ranking, attributes, photos, and reviews), *User* (profile, saved places, and recent views), and *Review* (rating and text). Venue and

review data flow from the FastAPI backend via REST API calls; filters and sorting are computed client-side. User authentication state, saved places, and recently viewed venues are synchronised with the backend, while theme preference persists in `localStorage`.

3) *User Interface Design*: The interface is organised into four complementary surfaces:

- *Left sidebar*: unified search bar, origin selector, type and crowdedness filters, sort controls, result cards, recent searches, and a chat shortcut.
- *Map*: interactive Manhattan map with clustered venue markers; Google Maps is used when an API key is provided, otherwise a stylised grid fallback preserves the layout.
- *Right panel*: switches between a venue detail view (photos, Quiet Score, attribute bars, busy level, opening hours, prediction chart, reviews, and save action) and a conversational chat panel.
- *Modals*: authentication, user profile, settings, and add-review dialogs.

A top navigation bar provides branding, weather, theme and language toggles, and user-profile access.

4) *Interaction Design*: Users search by name or address, filter by venue type and crowdedness level, and sort by ranking score or distance. Clicking a marker or card opens the venue detail panel and records the view. Authenticated users can save places, add reviews, and view their saved places and recently viewed list. Keyboard shortcuts (e.g., `/` for search, arrow keys to navigate the list, `Enter` to open, `Esc` to close) support efficient browsing.

5) *Visual Design*: The interface adopts a clean, modern aesthetic using Tailwind CSS: soft slate backgrounds, rounded corners, translucent glassmorphism panels, and a sky-to-teal accent gradient. Framer Motion provides smooth transitions for panel switching, modal appearance, and list updates. Light and dark themes are implemented through CSS custom properties, with the user's preference stored in `localStorage`.

6) *Localisation and Responsiveness*: The application supports English, Chinese, and Spanish through a React language context that loads translated content from the backend with bundled fallback translations. The layout is optimised for desktop viewports (1024px and above), with fixed minimum widths for side panels to maintain usability. Mobile responsiveness is identified as future work.

7) *Design Outcome*: The resulting design supports the core task flow—search, filter, explore, save, decide—within a single cohesive screen. Consistent interactive patterns, reinforced by motion, make the application feel polished while keeping quiet-space discovery as the central focus.

C. Backend Architecture

Hush-Hub follows a three-tier architecture: a React single-page application served to the client, a Python REST API layer, and a PostgreSQL relational database, all deployed on a Tencent Cloud virtual machine (Ubuntu 22.04).

The backend is implemented as a FastAPI application exposing a versioned REST API under the `/api` prefix. SQLAlchemy 2.0 serves as the ORM layer, with Pydantic v2 models handling request validation and response serialisation. Authentication is implemented with JWT bearer tokens using PyJWT, with `bcrypt` password hashing via `Passlib`. The following primary router modules structure the API:

- `/api/venues` — venue listing (with filtering by type, bounding box, and minimum quiet score), and individual venue detail with the full attribute-score breakdown and representative editorial quotes.
- `/api/venues/{id}/quiet-profile` — per-venue ML quiet profile returning today's hourly busyness curve, area-noise indicators, construction and event signals, and the indoor calm-bar breakdown.
- `/api/venues/{id}/live` — live crowdedness derived from the current-hour busyness percentage.
- `/api/chat` — AI assistant endpoint (authentication required), persisting conversation history and forwarding the last ten turns as context to the language model.
- `/api/auth`, `/api/users`, `/api/reviews` — user registration, login, favourites, saved lists, and venue reviews.

The PostgreSQL schema centres on a `venues` table keyed by a stable `canonical_id`, storing editorial meta-data (name, address, coordinates, venue type, quiet score, display rating, attribute scores as JSONB, and representative quotes). Three additional tables hold ML pipeline outputs: `venue_busyness_hourly`, `venue_indicators` (one row per venue with road-noise, 311-complaint, construction, and event features), and `venue_calm_bar` (editorial noise and crowding mention counts). User data is stored across `users`, `user_favorites`, `user_saved`, and `reviews` tables. Chat history is persisted in `chat_messages`, enabling cross-session conversation context.

The conversational assistant endpoint (POST `/api/chat`) requires user authentication, ensuring chat history can be persisted and conversation context maintained across sessions. On each request, the system: (i) parses natural-language constraints from the query (minimum/maximum quiet score, minimum rating, venue type) using regular expressions; (ii) queries the venue database with those constraints, randomising results when no score constraints are present to promote diversity; (iii) formats up to 15 matching venues as structured context; and (iv) forwards the context together with the last ten turns of conversation history to the Qianwen `qwen-turbo` model via the Alibaba Cloud DashScope OpenAI-compatible API. Both the user message and the assistant reply are saved to the `chat_messages` table, making conversation history available on subsequent sessions.

D. Frontend Web Application

The Hush-Hub desktop client is a single-page application (SPA) built with React 19 [10] and the Vite development and production toolchain [11]. Following SPA principles of

client-side rendering and dynamic interaction [12], the application provides dynamic updates without full-page reloads. The interface is organized as a three-pane workspace: a left sidebar for search and filtering, a central interactive map, and a right panel for venue details or conversational chat. The root App component is responsible for managing shared application state, such as the venue list, active filters, selected venue, authentication status, and UI theme. Feature UI is decomposed into reusable presentational components, while all HTTP access to the FastAPI backend is encapsulated in a dedicated `services/` layer (venues, auth, venueSignals, userReviews, and user lists).

a) Layout and navigation: The primary layout (MapsLayout) adopts a responsive CSS Grid design, consisting of a left sidebar for venue search and filtering, a central interactive map, and a slide-in right panel for venue details or AI chat. Scrolling is confined to the sidebar and detail panel, allowing the map to remain fixed within the viewport during user interaction. A top navigation bar provides access to branding, current weather, theme and language settings, user authentication, profile management, and the AI chat interface.

b) Google Maps integration: Interactive mapping is implemented using the Google Maps JavaScript API, accessed through the React wrapper `@vis.gl/react-google-maps` [13]. The required Google Maps libraries, including Advanced Markers and Places, are loaded during application startup. The map is centred on Manhattan and renders clustered venue markers using a custom viewport-based clustering implementation. The map colour scheme follows the active light or dark theme. Users can use geolocation to determine their current position, set the search origin through Google Place Autocomplete, and automatically pan or zoom when a venue is selected. If no Maps API key is configured, a non-interactive fallback map preserves the overall layout for local development.

c) Search, filtering, and sorting: The left sidebar provides search, filtering, and sorting functions to help users find suitable study venues. It supports both free-text search and Google Place Autocomplete [14]. When a place is selected from the autocomplete suggestions, the map recentres on that location. Free-text search matches venue names and addresses.

Users can filter venues by type (cafe, library, or public study area). Additional options are available under the “More filters” panel, including area busyness, indoor quietness, opening status, wheelchair accessibility, and minimum Google rating. The area-busyness filter uses the current-hour busyness level returned by the backend and allows users to display only venues with *low* or *medium* busyness. The indoor quietness filter distinguishes between reviewer-confirmed *calm* venues and those reported as *lively*.

Search results are sorted by distance whenever a search location or the user’s current GPS location is available. Otherwise, venues are ordered using the application’s ranking score.

d) Venue detail and environmental signals: Selecting a venue opens a detail panel displaying photographs, editorial

attributes, user reviews, save and directions actions, and a dedicated venue-signals section. The panel presents information on indoor quietness, nearby noise and disruption, together with a Busy Level forecast based on the venue’s hourly busyness profile. When a complete daily profile is available, the forecast is displayed as a 24-hour line chart. An optional map overlay visualises current busyness as a weighted heatmap, helping users compare activity levels across different areas.

e) Internationalisation, theme, and authentication: The interface supports English, Chinese, and Spanish through a React language context. Language resources are loaded with bundled fallback translations. The selected language is stored in `localStorage` and is also applied to the Google Maps interface. Light and dark themes are implemented using CSS custom properties with a `data-theme` attribute, following WCAG recommendations for colour contrast [15]. User authentication is based on JWT, with login and registration provided through an email/password modal. New accounts must be activated using a time-limited email verification link before personalised features such as saved venues, reviews, and recently viewed places become available. Password recovery is implemented using the same email-link mechanism.

E. Mobile Application

While the web-based Hush-Hub platform offers comprehensive search capabilities, users searching for quiet workspaces are often on the move in urban environments. Consequently, the mobile application was conceived not as a direct port of the web interface, but as an essential companion serving an entirely different product context. The web platform operates in a “Dashboard” mode, offering high information density (simultaneous maps, lists, and analytics) built for deep, stationary planning. In contrast, the mobile app operates in an “Immersive” mode designed to provide on-the-go access, real-time location-aware discovery, and immediate navigation capabilities. Because users need to find suitable quiet venues quickly wherever they are, the mobile business logic shifts away from heavy data analysis. Instead, it prioritizes intuition and rapid decision-making by leveraging clear visual cues and emotional user quotes rather than complex metric sliders.

1) Cross-Platform Integration Constraints: The primary challenge in developing the mobile companion was delivering a consistent, high-performance user experience across both iOS and Android devices without the overhead of maintaining two separate native codebases. Furthermore, the application required seamless integration of interactive maps, efficient real-time location tracking, and robust synchronization with the same core FastAPI backend services used by the web platform.

2) Architecture and Development Approach: To address these challenges, the application is built entirely on Flutter, which compiles to native ARM code. To prevent the codebase from deteriorating as it scales, we adopted a Feature-First Architecture, organizing code by business domains (e.g., `lib/features/auth`, `map`) to maintain high cohesion, supported by a centralized core layer (`lib/core/`) for

maximum reusability. The declarative UI composes complex screens from small, reactive widgets whose state is cleanly decoupled using the `provider` package. Declarative navigation and deep linking are handled via `go_router`. Network requests to our shared FastAPI backend are efficiently managed by `dio`, and localization is built-in by default (`lib/l10n/`), ensuring future global scalability without hardcoded strings.

3) *User Interface and Mobile-Friendly Engineering*: The implemented user interface achieves extreme high-fidelity to the original design mockups across its core screens: the Discovery Map, the immersive Place Details, a native AI Chat Assistant for conversational discovery, fully functional Auth/Login and Profile flows, and supplementary List views. State between these core screens is seamlessly preserved using a `StatefulShellRoute` indexed stack. To engineer a truly “mobile-friendly” experience tailored to navigating busy urban environments, the interface integrates several specific design optimizations:

- **Thumb-Friendly Navigation**: A frosted-glass floating bottom navigation bar and large touch targets optimize the UI for one-handed use;
- **Location-First Onboarding**: Using the `geolocator` package, the app polls GPS immediately to instantly render nearby spaces upon launch, dynamically calculating distances;
- **Mobile-Native Interactions**: The interface relies heavily on Bottom Sheets instead of jarring full-screen modals, allowing users to view place details while maintaining spatial context on the map;
- **Performance Optimizations**: Map rendering is enhanced with the `google_maps_cluster_manager_2` package, grouping map markers to prevent visual clutter and device overheating in dense areas like Manhattan;
- **Outdoor Accessibility**: The app leverages dynamic text scaling and native screen reader integration (iOS VoiceOver and Android TalkBack) to assist users navigating unpredictable city streets, as well as URL launching to seamlessly transition to system routing apps.

4) *Unified Backend Synchronization*: Despite the differing UX paradigms, the mobile application maintains strict data consistency with the web platform. Utilizing `dio` interceptors, the app securely manages authenticated session state by automatically retrieving JWT tokens from native `SharedPreferences`. This ensures that all machine-learning pipeline outputs, quietness scores, user favorites, and personalized lists remain seamlessly synchronized across both platforms.

F. Data Collection Methods

Hush-Hub draws on two independent bodies of data. The first is a curated venue database of Manhattan work-and-study spaces, built because no public dataset records how suitable a place is for focused work. The second is an environmental layer describing what is happening around each venue, assembled from public civic sources. Table II lists the sources and the per-venue features derived from them.

TABLE II
DATA SOURCES AND DERIVED PER-VENUE FEATURES

Source	Role	Volume	Derived feature
MTA Subway Hourly Ridership	Area busyness (primary)	12.4M rows → 1.54M station-hours	168-hour busyness curve [16]
NYC 311 Noise Complaints	Reported neighbourhood noise	4.4M rows	Noise complaints per year within 100 m [16]
DOB NOW Approved Permits	Construction disruption	234k rows	Active building sites within 150 m [16]
NYC Permitted Events	Transient street disruption	6,009 rows	Loud events within 250 m [16]
OpenStreetMap (Overpass)	Road network and venue attributes	Manhattan extract	Road traffic noise in dB(A) [17]
Google Places	Venue identity	1,018 lookups	Place identifier only [18]
Editorial corpus (this work)	Indoor workspace attributes	106 articles, 1,387 mentions	326 venues, ten attribute bars

Building the venue database. Generic review platforms were rejected because star ratings aggregate enjoyment across all use cases and poorly proxy suitability for focused work. Editorial “best places to work from” articles are better suited: journalists have already compared venues against our criterion and discuss Wi-Fi, power, noise and seating directly. A seven-stage pipeline turns these articles into structured records. Discovery yields 444 candidate URLs using a fixed query set and a review-site blocklist. Corpus extraction uses `Trafilatura` [19] to preserve heading structure. Relevance filtering with `Claude Haiku 4.5` produces 106 articles (after reinstating nine library guides that a café-shaped classifier had incorrectly rejected). Extraction emits one typed record per mention, with nine workspace attributes and a verbatim, source-checked quote. Canonicalisation resolves 1,018 distinct name-and-location pairs to Google `place_ids`, leaving 326 venues after merging duplicates and removing closed or out-of-scope entries. Aggregation stores attribute counts rather than averages to preserve evidence strength and disagreement, and enrichment adds an OpenStreetMap factual layer matched by location.

Turning civic records into venue features. Each civic dataset contains millions of dated, geolocated records; the application needs one row per venue. We use a Gaussian distance-decay kernel $w(d) = \exp(-d^2/2\sigma^2)$ where nearer events count more [20], choosing the spatial treatment per source. Construction sites and subway ridership use decay weighting; 311 complaints and active construction sites are counted within fixed radii because the counts are directly checkable. Road traffic noise uses a physical propagation model: each OpenStreetMap road class is assigned a level at 10 m [21], [22] and attenuated as $L(d) = L_0 - 10\gamma \log_{10}(d/D_0)$ with $\gamma = 1.0$ [23], [24]. A venue takes the loudest nearby road. Reported noise counts 311 complaints per year within 100 m; construction counts distinct active sites within 150 m [25]; permitted events are geocoded from free-text street ranges and classified as loud or quiet by closure type.

Presentation architecture. We originally combined signals into a single “quiet score” but abandoned it. The signals measure different things—road noise and 311 complaints are nearly uncorrelated ($\rho = -0.08$)—and there is no ground-truth quietness label, so any weighting would be arbitrary. Construction is ubiquitous and loud events are rare, so both dilute rather than discriminate. Because reviewers already experienced ambient street noise, adding a separate noise penalty would double-count. The application therefore reports four independent groups—quiet inside, area noise, happening nearby and busyness—each in interpretable units.

G. Machine Learning Pipeline

We framed busyness prediction as a supervised regression problem: predict hourly subway ridership at a station, then transfer that prediction to nearby venues. The raw MTA records were aggregated to 1,538,235 station-hour totals across 123 Manhattan station complexes from January 2025 to June 2026. Each station-hour is described by hour of day, day of week, weekend and month indicators, cyclical encodings of hour and day of year, public-holiday and adjacent-day flags, and three training-only summaries of how busy that station typically is overall, by hour, and by day of week. A chronological split held out the final six weeks for evaluation; random splitting would place future observations in the training set and would not test the forward-prediction claim.

The demanding baseline is the average ridership for that station, day-of-week and hour. For regular commuting patterns this is a strong predictor. Four models were trained on identical data using scikit-learn [26]; Table III reports mean absolute error (MAE) and root-mean-square error (RMSE) on the held-out period. The random forest is clearly best, reducing MAE by 19% and RMSE by 28% relative to the baseline. Gradient boosting improves only slightly on MAE but its lower RMSE shows it handles unusually busy hours better. Linear regression is substantially worse than the baseline, confirming that ridership patterns are non-linear and need a flexible model.

TABLE III
MODEL COMPARISON ON THE SIX-WEEK HELD-OUT PERIOD

Model	MAE	RMSE	vs. baseline
Baseline (cell mean)	87.7	210.1	—
Linear regression	202.9	484.0	−131.2%
HistGradientBoosting	84.4	171.0	+3.8%
Random forest	71.0	150.4	+19.1%

Station predictions are transferred to venues using Eq. (1) with $\sigma = 400$ m, approximately the distance people will walk to a station. A weighted sum rather than an average ensures that a venue far from any station receives a genuinely small value instead of being rescaled upward. This produces 54,768 venue-hour predictions ($326 \times 7 \times 24$). The results behave plausibly: Midtown peaks at the evening commute, waterfront venues sit near zero, and 94% of venue-hours are quiet on Sunday mornings. With only 17 months of post-pandemic

data, the model learns daily and weekly rhythms reliably, but holiday and seasonal effects are weak.

The venue pipeline resolved every mention to a confirmed identity, with the last nine ambiguous cases checked against street-level imagery, and 239 of the 326 venues (73%) carry an OpenStreetMap factual layer. Adding that layer raised the number of venues with accessibility information from 9 to 82, demonstrating the value of keeping factual and editorial sources separate. Busyness thresholds were set from the observed distribution, placing the low/medium and medium/high boundaries near the 75th and 95th percentiles of all venue-hours. This produces a 71.4% / 24.4% / 4.1% split, so the filter genuinely divides the corpus.

IV. EVALUATION

A. Functional Testing

Functional testing was carried out throughout development to verify that each module behaved according to the system requirements. Testing combined manual user-interface testing, API validation through the Swagger UI (`/docs`), regression testing after feature updates, and browser compatibility checks using Chrome and Edge. The 326 canonical venues imported into PostgreSQL served as the primary dataset.

Table IV covers the venue discovery module. Tests were repeated across multiple search terms, filter combinations, and sort modes to ensure consistency.

TABLE IV
VENUE DISCOVERY MODULE TESTS

ID	Description	Expected Result	Status
VD-01	Display venue list on load	326 venues loaded and rendered in sidebar	Pass
VD-02	Search venue by name	Matching venues shown; non-matching venues hidden	Pass
VD-03	Search venue by address	Venues near matching address displayed	Pass
VD-04	Filter by venue type	Only selected type (cafe/library/etc.) shown	Pass
VD-05	Filter by crowdedness level	Venues matching selected busy level shown	Pass
VD-07	Sort by distance	Venues ordered by distance from origin	Pass
VD-08	Click marker on map	Corresponding venue detail panel opens	Pass
VD-10	Empty search result	Empty state with guidance tips displayed	Pass

All eight discovery cases passed. One issue was fixed: keyboard focus occasionally did not scroll the selected card into view after rapid filtering. This was resolved by deferring the scroll-into-view effect until the filtered list re-rendered.

Table V covers the venue detail module, which displays enriched venue information and supports saving, reviewing, and route guidance.

All eight detail cases passed. The save toggle did not update the UI immediately when the backend request was slow; this was resolved by refreshing local state optimistically after clicking.

TABLE V
VENUE DETAIL MODULE TESTS

ID	Description	Expected Result	Status
VDT-01	Open venue detail panel	Photos, name, address, type, Quiet Score displayed	Pass
VDT-02	Display attribute bars	Noise, seating, Wi-Fi, power attributes shown	Pass
VDT-03	Display busy level badge	Current crowdedness indicator shown	Pass
VDT-05	Display prediction chart	Prediction chart rendered	Pass
VDT-06	Show venue reviews	Existing reviews listed with ratings	Pass
VDT-07	Save a venue	Venue appears in saved list	Pass
VDT-08	Remove saved venue	Venue removed from saved list	Pass
VDT-09	Open Google Maps directions	External Google Maps route opens in new tab	Pass

Table VI covers user authentication, saved places, recently viewed venues, and review submission.

TABLE VI
AUTHENTICATION AND USER LISTS MODULE TESTS

ID	Description	Expected Result	Status
AUTH-01	Register new account	Account created and user logged in	Pass
AUTH-02	Login with valid credentials	Session token stored and profile accessible	Pass
AUTH-03	Login with invalid credentials	Error message displayed	Pass
AUTH-04	Logout	Session cleared and auth state reset	Pass
AUTH-05	View saved places	Saved venues listed in profile panel	Pass
AUTH-06	View recently viewed	Last 5 viewed venues displayed	Pass
AUTH-07	Submit a review	Review stored and displayed in venue detail	Pass
AUTH-09	Session persistence	User remains logged in after page refresh	Pass

All eight authentication cases passed. API responses were verified using Swagger UI and browser Developer Tools. The login modal now opens automatically when an unauthenticated user attempts a protected action.

Table VII covers the weather, chat, prediction, and mobile modules. These features either depend on external APIs or were tested on the Flutter mobile client.

The weather module degraded gracefully when the OpenWeather key was missing. Chat and prediction returned placeholder data and await full ML/LLM integration. Mobile testing on an Android emulator and physical device confirmed map rendering, filter chips, venue detail navigation, and user-list synchronisation.

Table VIII summarises the main issues discovered during functional testing and how they were resolved. These fixes were verified by re-running the relevant test cases.

All five issues were resolved and their associated test cases were re-run successfully. The fixes improved responsiveness, mobile consistency, and robustness when external API keys are unavailable.

TABLE VII
WEATHER, CHAT, PREDICTION AND MOBILE TESTS

ID	Description	Expected Result	Status
WTH-01	Display current weather	Temperature, humidity, condition rendered	Pass
WTH-05	Missing API key	Weather panel shows placeholder gracefully	Pass
CHAT-01	Send chat message	Response returned without error	Pass
CHAT-02	Chat UI panel opens	Panel switches from detail to chat view	Pass
PRED-01	Request current Quiet Score	Score returned based on endorsement score	Pass
PRED-03	Display prediction chart	Chart rendered in venue detail	Pass
M-01	Launch mobile app and load map	Google Map renders with clustered markers	Pass
M-07	View profile and lists	User-specific lists load from backend	Pass

TABLE VIII
FUNCTIONAL TESTING ISSUES AND RESOLUTIONS

ID	Module	Issue	Resolution
ISS-01	Venue Discovery	Keyboard focus occasionally did not scroll the selected card into view after rapid filtering.	Deferred scroll-into-view until the filtered list re-rendered.
ISS-02	Venue Detail	Save toggle did not update the UI immediately when the backend request was slow.	Applied optimistic local-state update after clicking.
ISS-03	Mobile Search	Selecting a Google Places location panned the map but did not filter the venue list by proximity.	Adjusted visible-venue synchronisation logic so the list follows the searched location.
ISS-04	Mobile Links	Hyperlinks and backend requests failed after the development backend IP address changed.	Updated the base API URL and rebuilt the mobile client.
ISS-05	External APIs	Missing OpenWeather API key left the weather widget blank instead of showing a fallback.	Added a placeholder state that renders when the key is absent.

In total, 32 representative functional test cases were executed; all passed. The main user-facing features operate as intended. The main limitations are the lack of automated unit tests and the placeholder status of the AI components.

B. User Experience

The UX design was driven by the three personas identified in Section III-A: university students, remote office workers, and tourists. Each group has different primary needs, so the interface was organised around a single-screen, map-centric workflow that keeps search, spatial exploration, and detailed information in constant view.

Design rationale. A three-pane layout was chosen to minimise context switching. The left sidebar consolidates search, filters, and result cards, allowing students to compare candidates quickly without leaving the list. The central map provides the spatial awareness that tourists and remote workers need to judge proximity to hotels, campuses, or transit. The right panel switches between venue details and an AI chat assistant, keeping auxiliary information one click away rather than on a separate page. Glassmorphism panels, rounded corners, and a sky-to-teal accent gradient were selected to reduce visual noise and keep the focus on finding a quiet space

[15]. Framer Motion provides the transitions that reinforce the relationship between selecting a card, panning the map, and opening the detail panel.

User evaluation. Informal usability sessions were conducted with five representative users (two students, two remote workers, and one tourist). Each participant completed four tasks: find a cafe near a given address, filter for low crowdedness, save a venue, and submit a review. Completion times and verbal feedback were recorded. All participants completed the core tasks within two minutes. Positive feedback highlighted the map-centric layout, the speed of the filter chips, and the clarity of the Quiet Score and busy-level badges. The weather widget was cited by remote workers as a useful deciding factor for outdoor work. Two participants suggested richer comparison of multiple venues, which is already implemented as a side-by-side comparison panel. One tourist found the animated empty-state tips helpful when filters were too strict.

Accessibility and localisation. The interface supports English, Chinese, and Spanish through a React language context that loads translations from the backend. Light and dark themes are implemented with CSS custom properties and follow WCAG 2.2 contrast guidance [15]. Keyboard shortcuts (e.g., `/` for search, `Enter` to open a venue, `Esc` to close) were added for power users.

Limitations. The layout is optimised for desktop viewports (1024 px and above). Mobile responsiveness and touch interactions are identified as future work, to be supported by the planned mobile application.

C. System Performance

System performance was evaluated on the local deployment (Windows 11, Python 3.14, Node.js 24, PostgreSQL 16.9, Chrome/Edge) under normal single-user load.

Responsiveness. The React SPA loads in about 1.2 s and renders the first venue list within 2.0 s; client-side filtering and sorting complete in under 50 ms. Backend detail and quiet-profile calls return in 90–160 ms, and `GET /api/venues` returns 200 venues in 180–250 ms. Chat requests take 1.5–4 s because they forward context to the Qianwen API.

Database, prediction and stability. Venue and user tables are indexed on canonical ids, user ids and foreign keys; the 54,768-row `venue_busyness_hourly` table uses a composite index on `(canonical_id, dow, hour)`. Batch imports of 1.54 M station-hours finish in under three minutes; model training takes about 45 s and inference over all venue-hours completes in under 2 s. The FastAPI backend and PostgreSQL ran continuously without crashes, with the Uvicorn worker below 350 MB.

Mobile and model-serving performance. The Flutter mobile client cold-starts in under 2 s on a mid-range Android device and renders the initial map cluster within 1.5 s after the first `/api/venues` response. Marker clustering and heatmap overlay remain above 55 fps with 326 venues. The trained scikit-learn random-forest model is serialised with joblib and loaded once at backend startup; prediction for a single venue-hour completes in roughly 5 ms, so the 54,768-slot matrix

can be recomputed on demand in under 2 s if new civic data arrives. Caching is used sparingly: weather data is refreshed every 15 minutes and venue lists are cached for 5 minutes, avoiding repeated database scans while keeping displayed information current.

Resource usage and end-to-end latency. Memory usage remains modest: the frontend bundle is approximately 1.5 MB gzipped; the backend resident set stays below 350 MB with the full venue and busyness tables loaded. End-to-end latency from a user click to a rendered venue detail panel is typically 250–400 ms when the backend is warm. The slowest path is the chat endpoint, where network round-trips to the Qianwen API dominate; this is acceptable for an optional assistant feature but would need rate limiting and streaming in a production setting.

V. CONCLUSION

This paper presented Hush-Hub, a full-stack web application that helps users find quiet workspaces in Manhattan by fusing heterogeneous civic and editorial data into interpretable, time-varying signals. The system curates 326 venues from editorial “best places to work” articles, enriches them with road-traffic noise, 311 complaint density, construction and event disruption, and an ML-derived hourly busyness curve from MTA ridership, and serves the results through a FastAPI backend with a React map-centric frontend and a Flutter mobile companion.

The main contributions are threefold. First, we showed that editorial articles are a more suitable source than generic review platforms for the task-specific criterion of quietness, and we described a reproducible seven-stage pipeline that turns those articles into structured, evidence-backed venue records. Second, we demonstrated that keeping signals separate and reporting them in real-world units (decibels, complaints per year, site counts, percentages) is more honest and more useful than collapsing them into a single opaque score when no ground-truth quietness label exists. Third, we showed that a random-forest ridership model can beat a strong temporal baseline by 19% on MAE and that the predictions transfer plausibly to venues through a simple distance-decay kernel.

Hush-Hub meets its core functional goals: users can search, filter, save, review, and receive conversational venue recommendations, and the mobile companion extends discovery to on-the-go use. Testing confirmed that the venue discovery, detail, authentication, saved-list, and weather modules operate correctly.

Future work includes several directions. First, the conversational assistant can be upgraded with retrieval-augmented generation. Second, the ridership model can be improved by incorporating real-time feeds. Third, mobile push notifications can alert users when a saved venue becomes quiet, when a highly rated workspace opens nearby, or when weather conditions favour outdoor co-working options. Finally, extending the venue pipeline and prediction model to other dense urban districts would test whether the editorial-data and ridership-transfer approach generalises beyond Manhattan.

REFERENCES

- [1] C. Deng, X. Dong, H. Wang, W. Lin, H. Wen, J. Frazier, H. C. Ho, and L. Holmes, "A data-driven framework for walkability measurement with open data: A case study of triple cities, new york," *ISPRS International Journal of Geo-Information*, vol. 9, no. 1, p. 36, 2020.
- [2] M. Akhavan, "Third places for work: a multidisciplinary review of the literature on coworking spaces and maker spaces," in *New Workplaces—Location Patterns, Urban Effects and Development Trajectories: A Worldwide Investigation*, 2021, pp. 13–32.
- [3] World Health Organization, "Environmental noise guidelines for the european region," WHO Regional Office for Europe, Copenhagen, Denmark, Tech. Rep., 2018.
- [4] E. Murphy and E. A. King, *Environmental Noise Pollution: Noise Mapping, Public Health, and Policy*. Amsterdam, The Netherlands: Elsevier, 2014.
- [5] J. Bao, Y. Zheng, D. Wilkie, and M. Mokbel, "Recommendations in location-based social networks: a survey," *GeoInformatica*, vol. 19, no. 3, pp. 525–565, 2015.
- [6] M. Ye, X. Liu, and W. C. Lee, "Exploring social influence for recommendation: a generative model approach," in *Proc. 35th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2012, pp. 671–680.
- [7] Y. G. Matsinos, A. Tsaligopoulos, and C. Economou, "Identifying the quiet areas of a small urban setting: the case of mytilene," *Global NEST Journal*, vol. 19, no. 1, pp. 17–28, 2017.
- [8] M. De Meulder, "Mapping quiet areas and their values in finland," *International Journal of Environmental Research and Public Health*, vol. 16, p. 1, 2026.
- [9] Hush City Lab, "Hush city: A citizen science app for quiet areas," [Online]. Available: <https://opensourcesoundscapes.org/hush-city/>, accessed: Jul. 20, 2026.
- [10] Meta Platforms, Inc., "React documentation," [Online]. Available: <https://react.dev/>, 2026, accessed: Jul. 20, 2026.
- [11] E. You and Vite contributors, "Vite documentation," [Online]. Available: <https://vite.dev/guide/>, 2026, accessed: Jul. 20, 2026.
- [12] M. S. Mikowski and J. C. Powell, *Single Page Web Applications: JavaScript End-to-End*. Shelter Island, NY, USA: Manning, 2014.
- [13] vis.gl, "React google maps," [Online]. Available: <https://visgl.github.io/react-google-maps/>, 2026, accessed: Jul. 20, 2026.
- [14] Google, "Place autocomplete overview," [Online]. Available: <https://developers.google.com/maps/documentation/javascript/place-autocomplete-overview>, 2026, accessed: Jul. 20, 2026.
- [15] World Wide Web Consortium (W3C), "Web content accessibility guidelines (wcag) 2.2," [Online]. Available: <https://www.w3.org/TR/WCAG22/>, Oct 2023, accessed: Jul. 20, 2026.
- [16] City of New York and State of New York, "NYC open data and NY state open data portals," Online, 2026, available: <https://opendata.cityofnewyork.us> and <https://data.ny.gov>. Datasets: 5wq4-mkjj (MTA Subway Hourly Ridership: Beginning 2025), erm2-nwe9 (311 Service Requests from 2020 to Present), rbx6-tga4 (DOB NOW: Build — Approved Permits), tvpp-9vvx (NYC Permitted Event Information) (accessed Aug. 1, 2026).
- [17] OpenStreetMap contributors, "OpenStreetMap," Online, 2026, available: <https://www.openstreetmap.org> (accessed 2026). Licensed under the Open Database License (ODbL).
- [18] Google, "Google maps platform terms of service," Online, 2026, available: <https://cloud.google.com/maps-platform/terms> (accessed 2026).
- [19] A. Barbaresi, "Trafilatura: A web scraping library and command-line tool for text discovery and extraction," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Aug. 2021, pp. 122–131.
- [20] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, ser. Monographs on Statistics and Applied Probability. London, UK: Chapman and Hall, 1986.
- [21] J. Almatawah and H. Matar, "Assessment of traffic noise level: A case study of a residential neighbourhood," *International Journal of Civil Infrastructure*, vol. 8, pp. 18–27, 2025.
- [22] J. Staab, A. Schady, M. Weigand, T. Lakes, and H. Taubenböck, "Predicting traffic noise using land-use regression—a scalable approach," *Journal of Exposure Science and Environmental Epidemiology*, vol. 32, no. 2, pp. 232–243, 2022.
- [23] International Organization for Standardization, *ISO 9613-2:2024 Acoustics — Attenuation of Sound During Propagation Outdoors — Part 2: Engineering Method for the Prediction of Sound Pressure Levels Outdoors*, ISO Std., 2024.
- [24] UK Department of Transport and Welsh Office, *Calculation of Road Traffic Noise*. London, UK: Her Majesty's Stationery Office, 1988.
- [25] Federal Highway Administration, "FHWA roadway construction noise model user's guide," U.S. Department of Transportation, Washington, DC, USA, Tech. Rep. FHWA-HEP-05-054, 2006.
- [26] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.